

Extending Kubernetes Self-Healing for Networking Failure

Arpan Gupta
24110051@iitgn.ac.in

Buha Deep Maheshbhai
24110082@iitgn.ac.in

Param Tanna
param.tanna@iitgn.ac.in

Ramji Purwar
ramji.purwar@iitgn.ac.in

Rathod Yuvraj Rajubhai
yuvraj.rathod@iitgn.ac.in

Solanki Viraj Rajeshbhai
viraj.solanki@iitgn.ac.in

Abstract—Kubernetes provides robust self-healing for container lifecycle events such as pod crashes and node failures; however, it remains fundamentally blind to network-layer degradation including Container Network Interface (CNI) plugin failures, DNS service disruptions, NetworkPolicy enforcement drift, and silent overlay tunnel breakages. These network failures cause cascading application outages despite all pods appearing healthy to the orchestrator. We approach the problem from a networking standpoint: each fault is a failure at a specific layer of the protocol stack—veth interfaces and ARP at the link layer, IPAM addressing and IP-in-IP/VXLAN overlay routing at the network layer, ICMP/TCP reachability at the transport layer, and DNS name resolution at the application layer—rather than a purely operational incident. In this paper, we present our custom Kubernetes operator that continuously monitors and auto-remediates four critical network subsystems: CNI health (Calico), CoreDNS availability, NetworkPolicy dataplane enforcement, and pod-to-pod connectivity. The operator employs a modular *Check* $\rightarrow$ *Evaluate* $\rightarrow$ *Remediate* pipeline architecture, where each module independently gathers network telemetry signals, applies threshold-based and triangulation-based failure classification, and executes targeted corrective actions via the Kubernetes API. For connectivity monitoring, we adapt the Pingmesh ring topology to achieve $O(N)$ probe scalability combined with a three-tier triangulation engine for root-cause isolation. The system is driven by a single cluster-scoped Custom Resource Definition (CRD) that exposes configurable detection thresholds and safety guardrails. We validate our operator across several fault-injection experiments on a multi-node Minikube cluster with Calico CNI, demonstrating automated recovery from CNI agent crashes, IPAM pool exhaustion, DNS latency spikes, corrupted upstream configurations, stale firewall rules, and broken overlay tunnels.

I. INTRODUCTION

A. Background and Motivation

Kubernetes has emerged as the de facto orchestration platform for containerized microservices [?]. Its built-in self-healing mechanisms—liveness probes, readiness probes, and deployment controllers—effectively handle container-level failures such as process crashes, out-of-memory kills, and node evictions. However, these mechanisms operate strictly at the *container lifecycle* layer and have no visibility into the network data plane.

In production Kubernetes clusters, the network fabric is a complex stack comprising: (i) a Container Network Interface (CNI) plugin (e.g., Calico, Cilium) that manages IP address allocation, virtual ethernet (veth) interfaces, and inter-node

overlay tunnels [?], [?]; (ii) CoreDNS for cluster-internal service discovery and external name resolution [?]; and (iii) NetworkPolicy resources whose enforcement is delegated to the CNI’s dataplane agent (e.g., Calico’s Felix daemon) via iptables or eBPF rules.

When any component in this stack degrades—a CNI agent crashes, DNS latency spikes, or an IP-in-IP tunnel interface drops—Kubernetes continues to report pods as *Running* (1/1 *Ready*) and nodes as *Ready*. Application traffic silently fails with connection timeouts, DNS resolution errors, or unauthorized packet leakage, leading to cascading service disruptions without any automated recovery.

Framed in the vocabulary of computer networks, the faults we target are not operational curiosities but failures spread across the entire protocol stack. At the *link layer*, each pod attaches to its host through a virtual Ethernet (veth) pair and resolves on-node addresses over ARP, so a downed veth or corrupted bridge breaks L2 frame delivery while the pod process stays alive. At the *network layer*, the CNI’s IP Address Management (IPAM) assigns every pod an IP, and cross-node traffic rides an overlay in which Calico encapsulates the original pod-to-pod packet inside an outer IP header using IP-in-IP or VXLAN; an exhausted IPAM pool or a dropped tunnel interface (tunl0) thus manifests as a pure L3 addressing and routing failure. At the *transport layer*, reachability is exercised with ICMP echo and TCP connection probes—the same primitives behind *ping* and a port check. At the *application layer*, service discovery depends on DNS: pods resolve names by sending UDP queries to CoreDNS on port 53, which answers cluster-internal names authoritatively and forwards external ones to an upstream resolver. NetworkPolicy enforcement is, at bottom, stateful packet filtering, with Calico’s Felix agent compiling each policy into Linux *netfilter* (iptables/ipset) rules that match on L3/L4 header fields. Our operator is therefore best understood not as a container-management add-on but as a system that watches and repairs faults along the full path a packet travels—from the veth interface, through IP routing and the overlay tunnel, to DNS resolution at the top of the stack.

B. Existing Solutions and Their Limitations

Several tools address aspects of Kubernetes network observability, but none provide continuous, closed-loop self-healing

across the full network stack:

- **Native Kubernetes:** DaemonSet controllers restart crashed CNI pods, but Kubernetes does not monitor IPAM capacity, DNS query latency, policy enforcement fidelity, or inter-node tunnel health [?].
- **Goldpinger [?]:** Deploys a full-mesh ping agent across nodes for connectivity visualization but performs no automated remediation and scales at $O(N^2)$.
- **Microsoft Pingmesh [?]:** Pioneered data-center-scale latency measurement using controller-orchestrated probing but was designed for bare-metal switches, not Kubernetes overlay networks, and does not include remediation logic.
- **NetBouncer [?]:** Provides active device and link failure localization in data center networks but operates below the Kubernetes abstraction layer.

C. Contributions

This paper makes the following contributions:

- 1) A **modular operator architecture** with a unified *Check* $\rightarrow$ *Evaluate* $\rightarrow$ *Remediate* pipeline that covers four network subsystems: CNI, CoreDNS, NetworkPolicy, and pod-to-pod connectivity.
- 2) An $O(N)$ **Pingmesh-inspired ring probing topology** with three-tier triangulation for root-cause isolation of inter-node and intra-node network failures.
- 3) A **CRD-driven configuration model** with per-module safety guardrails (auto-heal toggles, cooldown windows, escalation limits) enabling both autonomous and audit-only deployment modes.
- 4) **Comprehensive validation** across several fault-injection experiments demonstrating automated recovery from real-world network failure scenarios.

II. LIMITATIONS OF KUBERNETES NETWORK SELF-HEALING

Kubernetes was designed to manage container lifecycles, not network infrastructure health. This section identifies four categories of network failures that Kubernetes cannot detect or recover from autonomously.

A. CNI Plugin Failures

The CNI plugin (e.g., Calico) is responsible for pod IP allocation via IPAM, veth pair creation, and BGP routing mesh establishment [?], [?]. When the `calico-node` DaemonSet agent crashes or becomes unresponsive:

- New pods remain stuck in `ContainerCreating` state indefinitely, as the kubelet cannot invoke the CNI binary to configure the pod's network namespace.
- The node may be marked `NotReady` if the kubelet's network plugin status check fails, but Kubernetes takes no action to restart the CNI agent or recover IP allocations.
- IPAM pool exhaustion (all IPs allocated in a subnet) causes all new pod scheduling to fail with "No IPs available in pools", yet the Kubernetes scheduler has zero awareness of IPAM capacity.

- An inadvertently disabled Calico IPPool (`spec.disabled: true`) silently blocks all new IP allocations cluster-wide.

B. CoreDNS Failures

CoreDNS provides the foundational DNS resolution service for Kubernetes service discovery [?]. Native Kubernetes manages CoreDNS pod lifecycle but is blind to:

- **Latency degradation:** CPU throttling can spike DNS query round-trip times from ~ 2 ms to >1000 ms while pods remain `Running/Ready`. Kubernetes has no metric for application-layer DNS query duration.
- **Upstream misconfiguration:** Corrupted forward directives in the Corefile ConfigMap break external name resolution while internal `*.svc.cluster.local` resolution appears healthy.
- **Scale-to-zero:** If an operator or CI script accidentally scales CoreDNS replicas to zero, Kubernetes respects the desired state and leaves cluster DNS completely broken.

C. NetworkPolicy Enforcement Drift

Kubernetes stores NetworkPolicy resources declaratively in etcd but delegates enforcement entirely to the CNI's dataplane agent [?]. In Calico, the Felix daemon translates policies into Linux kernel `iptables/ipset` rules. If Felix crashes or enters `CrashLoopBackOff`:

- The kernel packet-filtering rules freeze in their last-known state.
- New policy creations, updates, or deletions are silently ignored by the kernel.
- `kubectl get networkpolicy` continues to show policies as active, creating a false sense of security-traffic that should be blocked remains permitted.

D. Pod-to-Pod Connectivity Failures

Pod-to-pod communication traverses virtual interfaces (veth pairs), host bridges, and overlay tunnels (IPIP/VXLAN) [?]. Kubernetes readiness probes only verify that a local process responds on a port—they do not validate that the virtual network fabric can deliver cross-node or intra-node traffic. When a veth interface is administratively downed, an IP-in-IP tunnel crashes, or `iptables FORWARD` rules are corrupted:

- Pods remain `Running (1/1 Ready)` on both the source and destination nodes.
- Application requests time out silently with no Kubernetes event or alert.
- Kubernetes has no mechanism for active network path probing or topology-aware failure detection.

III. SYSTEM ARCHITECTURE AND DESIGN

A. Operator and Custom Resource Definition

Our operator is implemented as a Kubernetes operator following the controller pattern [?], [?]. The system is governed by a single cluster-scoped Custom Resource Definition (CRD), `NetworkRemediation` (API version `v1alpha1`), which

TABLE I
CRD MODULE CONFIGURATION SUMMARY

Module	Key Parameters	Defaults
CNI	stuckPodThresholdSec	60 s
	ipamUsageThreshold%	80%
	calicoNodeLabel	k8s-app=calico-node
CoreDNS	expectedReplicas	2
	latencyThresholdMs	150 ms
	autoHealReplicas	true
	autoHealConfigMap	true
NetworkPolicy	autoHeal	true
	cooldownSeconds	60 s
Pod Connectivity	targetNamespace	default
	consecutiveFailThreshold	3
	autoRemediationEnabled	true
	maxRestartAttempts	2

serves as the declarative specification for all monitoring and remediation behavior.

The CRD specification is decomposed into four independent module configurations: `CNISpec`, `CoreDNSSpec`, `NetworkPolicySpec`, and `PodConnectivitySpec`. Each module specification exposes: (i) an enabled toggle for independent activation; (ii) detection thresholds and intervals; and (iii) safety guardrails controlling whether the operator may perform mutating actions or operate in audit-only mode.

Table I summarizes the key configurable parameters for each module.

B. Three-Phase Pipeline Architecture

The core design principle of our operator is the separation of concerns across three distinct operational phases, implemented as a common `Module` interface that all four network subsystem modules adhere to:

- 1) **Check Phase (Signal Gathering):** Collects raw network telemetry-pod statuses, DaemonSet health, IPAM allocations, DNS query latencies, active probe results-without making any decisions.
- 2) **Evaluate Phase (Failure Classification):** Analyzes gathered signals against configurable thresholds to classify severity (*info*, *warning*, *critical*) and determine whether automated remediation is warranted. This phase prevents false positives by requiring sustained threshold breaches.
- 3) **Remediate Phase (Corrective Action):** Executes the minimum necessary Kubernetes API mutations to restore network health-pod deletions (triggering DaemonSet respawns), deployment scaling, ConfigMap patches, node taints, or workload evictions.

Fig. 1 illustrates the three-phase pipeline within the reconciliation loop.

C. Dispatcher and Module Registration

The central reconciler acts as a *dispatcher*. On each reconciliation cycle, it: (i) fetches the `NetworkRemediation`

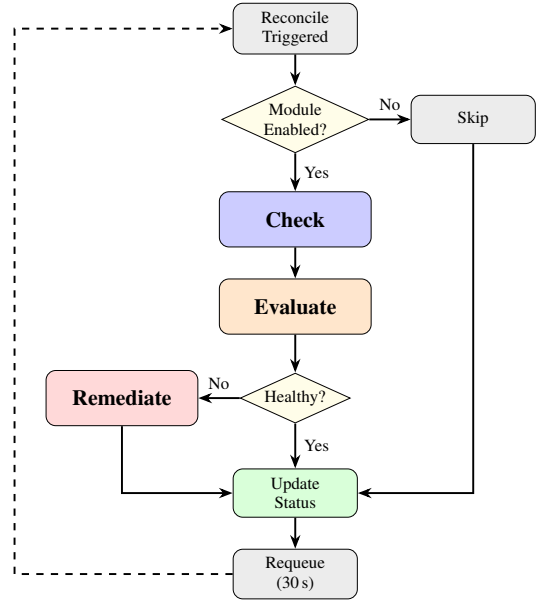


Fig. 1. Check → Evaluate → Remediate pipeline within the reconciliation loop. Each module runs this pipeline independently every 30 s.

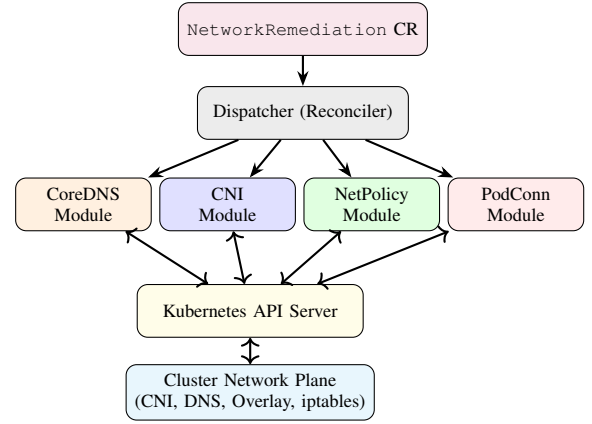


Fig. 2. System architecture. The dispatcher reads the CRD and runs each module’s pipeline. Modules interact with the Kubernetes API for telemetry and remediation.

custom resource; (ii) iterates through all registered modules; (iii) skips disabled modules; (iv) executes the Check → Evaluate → Remediate pipeline for each enabled module; (v) updates per-module status fields in the CR; and (vi) requeues for periodic re-check after 30 seconds.

This design ensures complete modularity—each network subsystem module operates independently, can be developed separately, and can be toggled on or off at runtime without affecting other modules. Fig. 2 shows the overall system architecture.

IV. DETECTION AND REMEDIATION POLICIES

This section describes the detection signals, evaluation heuristics, and remediation actions for each network subsystem module.

A. CNI Module

This module guards the link- and network-layer plumbing every pod depends on before it can send a single packet: the veth pair that connects it to the host, the IPAM allocation that gives it an L3 address, and the BGP/overlay routing mesh that carries its traffic between nodes.

1) *Detection Signals*: The CNI module continuously gathers four categories of signals:

- 1) **DaemonSet Health**: Compares `desiredNumberScheduled` against `numberReady` on the `calico-node` `DaemonSet`. Any mismatch triggers enumeration of unready pods.
- 2) **Stuck Workload Pods**: Scans all cluster pods in `Pending` phase with empty `podIP`. Pods exceeding the configurable stuck threshold (default: 60s) are flagged.
- 3) **CNI Event Correlation**: Queries Kubernetes warning events for `FailedCreatePodSandBox` messages containing CNI/IPAM error signatures.
- 4) **IPAM Capacity**: Queries `Calico IPPool` and `IPAMBlock CRDs` to detect disabled pools (`spec.disabled: true`) and exhausted blocks (100% utilization).

2) *Evaluation Heuristics*: The evaluator applies a strict priority ordering to avoid conflating symptoms with root causes:

- 1) *Unready calico-node agents* → critical severity, action: restart agents.
- 2) *Disabled IPPool with stuck pods* → critical, action: re-enable pool.
- 3) *Stuck pods with CNI errors* → critical, action: evict stuck pods.
- 4) *IPAM usage ≥ threshold* → warning, no disruptive action (log only).

3) *Remediation Actions*: Three targeted remediation branches are executed based on the evaluation:

- **Restart CNI Agents**: Deletes unready `calico-node` pods; the `DaemonSet` controller immediately respawns fresh agents that re-establish BGP routing and veth interfaces.
- **Re-enable IPPool**: Patches `spec.disabled` to `false` on affected `Calico IPPools`, unblocking IP allocations.
- **Evict Stuck Pods**: Deletes workload pods stranded in `ContainerCreating` to force rescheduling with clean network namespace initialization.

B. CoreDNS Module

DNS is the application-layer directory of the cluster: every service call begins with a name lookup. This module keeps that resolution path healthy, watching both the internal query service (UDP to CoreDNS on port 53) and the upstream forwarding used to resolve external names.

1) *Detection Signals*: The CoreDNS module monitors five aspects of DNS service health:

- 1) **Replica Availability**: Inspects the `coredns Deployment` for `availableReplicas` vs. `expectedReplicas` (default: 2).

2) **Pod Readiness**: Identifies pods in `CrashLoopBackOff`, `Error`, or `unready` states.

3) **DNS Query Latency**: Queries Prometheus for average request duration over a 1-minute window:

$$\text{Latency} = \frac{\text{rate}(\text{sum}[1m])}{\text{rate}(\text{count}[1m])} \quad (1)$$

4) **Upstream Configuration**: Parses the `Corefile ConfigMap` to validate forward directive targets against known-bad subnets (RFC 5737 [?]: 192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24).

5) **Resource Starvation**: Detects CPU limits ≤5m that cause latency spikes despite pods appearing healthy.

2) *Remediation Actions*:

- **Scale Recovery**: Patches the `CoreDNS Deployment` replicas back to `expectedReplicas`.
- **Pod Restart**: Deletes unready/crashlooping pods to trigger clean respawns.
- **ConfigMap Repair**: Replaces corrupted forward targets with fallback servers (default: `/etc/resolv.conf`, 8.8.8.8) and triggers a rolling restart.
- **Resource Restoration**: Resets CPU limits to baseline (100m) when throttling is detected.

Each remediation type is gated by independent safety toggles (`autoHealReplicas`, `autoHealConfigMap`) and a 30s cooldown window to prevent restart thrashing.

C. NetworkPolicy Module

A `NetworkPolicy` is, in networking terms, a stateful packet-filter rule matched on L3/L4 header fields (source/destination IP, port, protocol). Kubernetes only stores this intent; the CNI must program it into the kernel firewall. This module verifies that the enforcement path is actually alive, so that the rules a packet is checked against reflect the declared policy.

1) *Detection Signals*: The `NetworkPolicy` module targets the *control-plane-to-dataplane disconnect* described in Section III-C. It audits all `calico-node` pods in `kube-system` for:

- Pod phase ≠ `Running`
- Container readiness = `false`
- Container restart count ≥ 3
- Container state: `CrashLoopBackOff` or `Error`

Any pod matching these criteria indicates that the `Felix` enforcement agent is not actively programming kernel packet-filtering rules, leaving the dataplane in a stale state.

2) *Remediation Actions*: When unhealthy enforcement agents are detected and `autoHeal` is enabled, the operator deletes the degraded `calico-node` pod. The `DaemonSet` controller respawns a fresh pod, and on startup, `Felix` reads all declared policies from the Kubernetes API and programs `iptables/ipset` chains from scratch, clearing stale rules. A configurable cooldown (default: 60s) rate-limits consecutive restarts to prevent control plane churn.

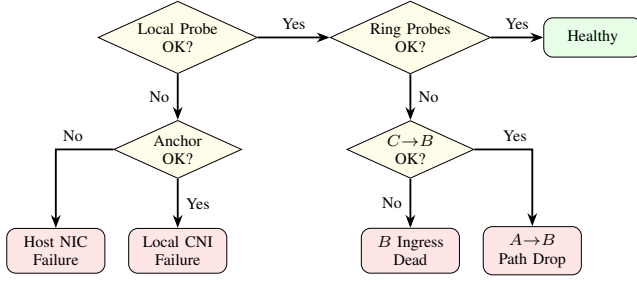


Fig. 3. Three-tier triangulation decision tree for root-cause isolation of connectivity failures.

D. Pod-to-Pod Connectivity Module

This module works at the network and transport layers, actively testing whether packets can actually traverse the veth, bridge, and overlay-tunnel path between pods-something readiness probes, which only check a local process, never verify.

1) *Probing Topology*: We adapt the Pingmesh [?] approach to Kubernetes by establishing a deterministic ring topology across N worker nodes. Nodes are sorted lexicographically and connected as:

$$\text{Forward: } Node_i \rightarrow Node_{(i+1) \bmod N} \quad (2)$$

$$\text{Reverse: } Node_i \rightarrow Node_{(i-1+N) \bmod N} \quad (N \geq 3) \quad (3)$$

This yields exactly $2N$ inter-node probes, achieving $O(N)$ scalability versus the $O(N^2)$ cost of full-mesh probing. Each probe is a lightweight ICMP echo or TCP connection attempt to the target pod's IP address, so a failed probe directly signals loss of L3/L4 reachability along that specific path. Each node additionally undergoes an $O(1)$ intra-node canary validation to verify local veth, bridge, and iptables FORWARD chain integrity.

2) *Three-Tier Triangulation*: The evaluate phase employs a three-tier pattern analysis engine to isolate root causes and prevent false remediation:

Tier 1 - Local CNI Validation: If a node's local probe fails but its system anchors (gateway, CoreDNS) are reachable, the root cause is isolated to the local CNI subsystem (dropped veth, corrupted eBPF map).

Tier 2 - Inter-Node Triangulation: When $Node_A \not\rightarrow Node_B$, the operator checks if $Node_C \rightarrow Node_B$ succeeds. If all peers fail to reach $Node_B$, the ingress network stack on $Node_B$ is declared dead. If only $Node_A$ fails, the issue is an asymmetric path/tunnel drop.

Tier 3 - Anchor Corroboration: For small clusters ($N < 3$) where triangulation is insufficient, probes against high-availability anchors (default gateway, kube-apiserver) disambiguate host-level versus tunnel-level failures.

Fig. 3 illustrates the three-tier triangulation decision tree.

3) *Hierarchical Remediation Policy*: Remediation escalates through three tiers of increasing blast radius:

Tier 1 - CNI Agent Restart: Deletes the `calico-node` pod on the affected node. The DaemonSet respawns the agent,

TABLE II
EXPERIMENTAL ENVIRONMENT SPECIFICATIONS

Component	Configuration
Cluster	Minikube (Docker driver), 2 nodes
Resources per Node	2 vCPUs, 4096 MB RAM
CNI Plugin	Calico (calico-node DaemonSet)
DNS Server	CoreDNS (2 replicas, kube-system)
Kubernetes Version	v1.31+
Operator Framework	Kubebuilder v4.15+, Go 1.24+
Test Workloads	backend-api, frontend-probe, dns-checker
Replicas per Workload	2 (spread across both nodes)

TABLE III
SUMMARY OF FAULT-INJECTION EXPERIMENTS

ID	Module	Fault Injected	Healed
E1	CNI	calico-node agent crash	✓
E2	CNI	IPAM pool exhaustion	✓
E3	CNI	IPPool disabled	✓
E4	CoreDNS	Scale replicas to zero	✓
E5	CoreDNS	CPU throttle (≤ 5 m)	✓
E6	CoreDNS	Corrupted upstream forward	✓
E7	CoreDNS	Pod CrashLoopBackOff	✓
E8	NetPolicy	Felix agent crash	✓
E9	PodConn	veth interface down	✓
E10	PodConn	IPIP tunnel crash	✓
E11	PodConn	iptables FORWARD DROP	✓
E12	PodConn	Inter-node overlay break	✓

which flushes stale routing tables, resets tunnel interfaces (tunl0, vxlan.calico), and rebinds veth pairs. Limited to `maxRestartAttempts` (default: 2).

Tier 2 - Node Taint and Cordon: If CNI restarts fail, the operator patches the Kubernetes Node resource with a `network-degraded:NoSchedule` taint and sets `spec.unschedulable = true`, preventing new workload placement on the degraded host.

Tier 3 - Workload Drain: As a last resort, the operator safely evicts all non-DaemonSet pods using the Kubernetes Eviction API, respecting `PodDisruptionBudgets`. Workloads are rescheduled onto healthy nodes by their parent controllers.

V. EXPERIMENTAL SETUP AND VALIDATION

A. Cluster Environment

All experiments are conducted on a multi-node Minikube cluster configured as shown in Table II.

B. Fault Injection Methodology

Each experiment follows a three-phase methodology: (i) establish a healthy cluster baseline; (ii) inject a specific network fault and observe native Kubernetes behavior (no operator); (iii) enable the operator and verify automated detection and recovery.

Table III summarizes the fault-injection experiments conducted across all four modules.

C. Validation Results

Across all experiments, the operator successfully detected the injected network failure within the first reconciliation cycle (30 s) and executed the appropriate remediation action. Key observations include:

- **CNI recovery:** Unready calico-node pods were restarted within 30–60 s, with BGP routing mesh convergence completing within an additional 10–15 s.
- **DNS restoration:** Scale-to-zero recovery restored full DNS availability within one reconciliation cycle. Corrupted upstream configurations were repaired and pods rolling-restarted within 45 s.
- **Policy resynchronization:** Crashed Felix agents were replaced within 30 s; kernel iptables rules were fully reprogrammed by the respawned agent within 5–10 s of pod startup.
- **Connectivity recovery:** The Pingmesh ring detected overlay tunnel failures within a single probe interval (10 s), and CNI agent restarts restored cross-node connectivity within 30–60 s.

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

We presented our custom Kubernetes operator that extends the platform’s self-healing capabilities to the network data plane. By implementing a modular Check → Evaluate → Remediate pipeline across four critical network subsystems—CNI, CoreDNS, NetworkPolicy enforcement, and pod-to-pod connectivity—our operator detects and auto-remediates network failures that are invisible to native Kubernetes mechanisms. By reasoning explicitly in terms of the protocol layers—L2 veth interfaces, L3 addressing and overlay routing, L4 reachability, and L7 name resolution—the operator localizes and repairs faults along the entire path a packet takes, rather than treating them as opaque application errors.

The operator’s $O(N)$ Pingmesh-inspired ring probing topology and three-tier triangulation engine enable scalable, accurate root-cause isolation of connectivity failures without the $O(N^2)$ overhead of full-mesh probing. The CRD-driven configuration model provides operators with fine-grained control over detection thresholds, safety guardrails, and remediation escalation, supporting both fully autonomous and audit-only deployment modes.

Validation across all fault-injection experiments confirmed that our operator achieves automated recovery from diverse real-world network failures including CNI agent crashes, IPAM exhaustion, DNS degradation, stale firewall rules, and broken overlay tunnels, typically within 30–60 s of fault occurrence.

B. Future Work

Several directions remain for future development:

- **Alertmanager Integration:** Connecting operator events to Prometheus Alertmanager for external notification channels (PagerDuty, Slack, email).

- **eBPF-Based Probing:** Replacing TCP/ICMP probes with eBPF kernel-level probes for deeper, lower-overhead CNI introspection.
- **Machine Learning Anomaly Detection:** Training time-series models on DNS latency and probe telemetry to detect subtle degradation patterns before threshold breaches.
- **Multi-Cluster Federation:** Extending the operator to monitor and heal cross-cluster networking in federated Kubernetes environments.
- **Helm Chart and OLM Packaging:** Publishing the operator via Helm charts and the Operator Lifecycle Manager catalog for production deployment.

REFERENCES